DATA 260 Homework 6

Farzaan Dalwai

https://github.com/farzaandalwai/data260-0686

Repository link verified working on 2026-10-17. The repository is public and its default branch is main. A read-only collaborator check found only the owner and no pending invitations; therefore direct collaborator access for **Sbnikitha** and **supriyaselvanganesan** is **not confirmed**. No access change was made.

Value	Result	Value	Result
SID4	0686	PORT_BASE	8686
PREFIX	s0686	SEED	686
VERIFY_SEED	260686	DOMAIN_ID	6 — Rental Housing Listings
Hardware	Apple M4, 10 cores	Memory	24 GB
Local model	llama3.2:latest	Embedding	all-MiniLM-L6-v2, 384 dims
Implementation commit	8130bc6f753f	Proposed tag	hw6 — not created pending approval

The implementation/evidence hash above is the fixed local commit tested by this report. The eventual submission tag must target the final report commit after review; no push or tag was performed.

Part 1 — MongoDB Document API

The new document view uses database **s0686_doc** and collection **rental_documents**. Pydantic rejects bad requests before database writes, while a strict MongoDB $jsonSchema validator independently protects stored data. Timestamps are timezone-aware and automatic. The stable evidence record remains present; temporary test records are deleted.

CRUD and validation evidence

```
@router.post('', status_code=201)
def create_document_listing(payload: RentalDocumentCreate): ...
@router.get('/{listing_id}')
@router.put('/{listing_id}')
@router.delete('/{listing_id}')
# Pydantic: title 1..100, enums, datetime, extra='forbid'
```

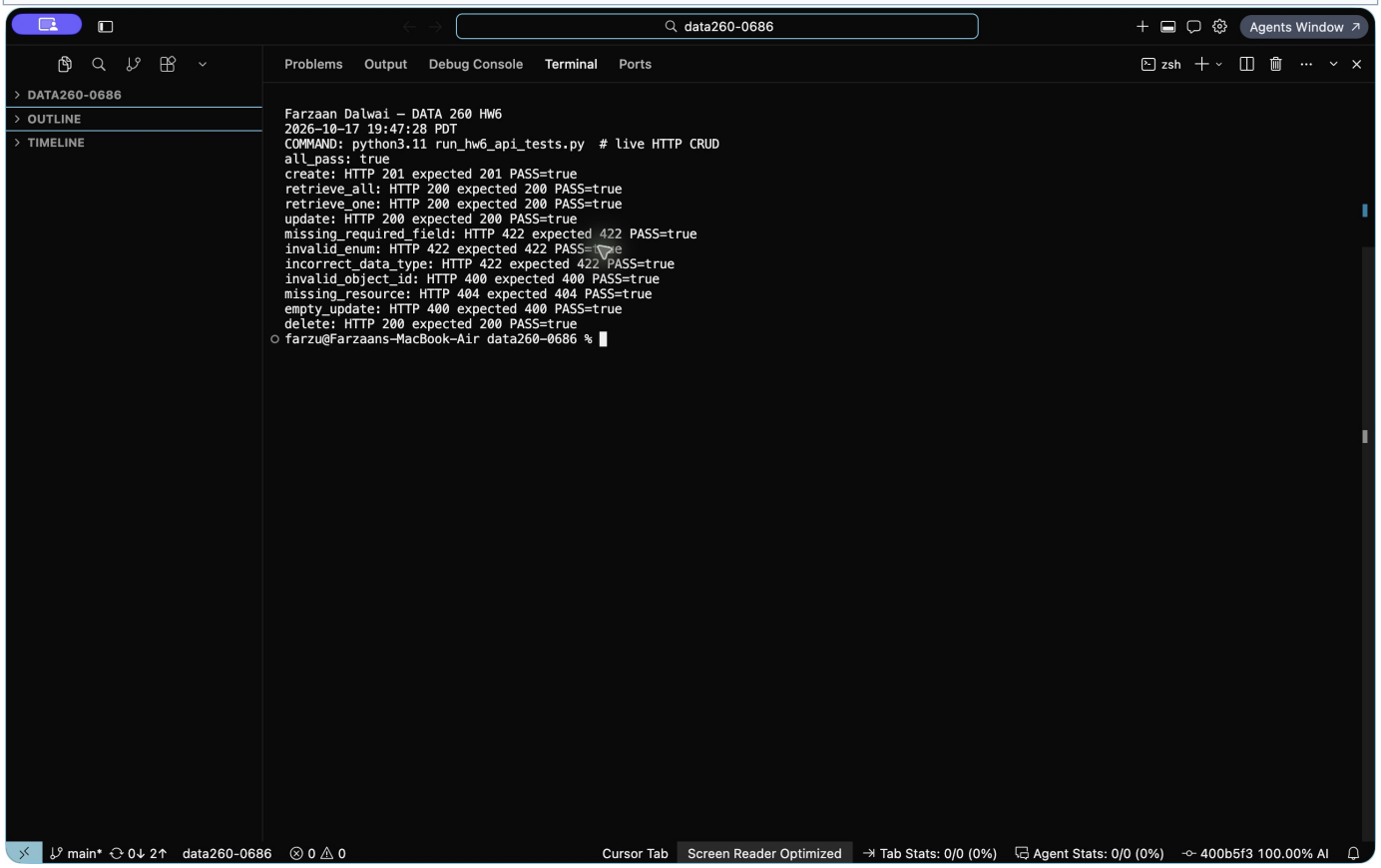


Figure 1. Live HTTP CRUD and validation: all_pass=true, 11/11 expected statuses.

MongoDB sample document

```
VALIDATORS['rental_documents'] = {
  '$jsonSchema': {'required': ['title','status','property_type',
    'available_from','created_at','updated_at'],
  'properties': {'title': {'maxLength': 100},
    'status': {'enum': ['available','pending','rented']}}}}
```

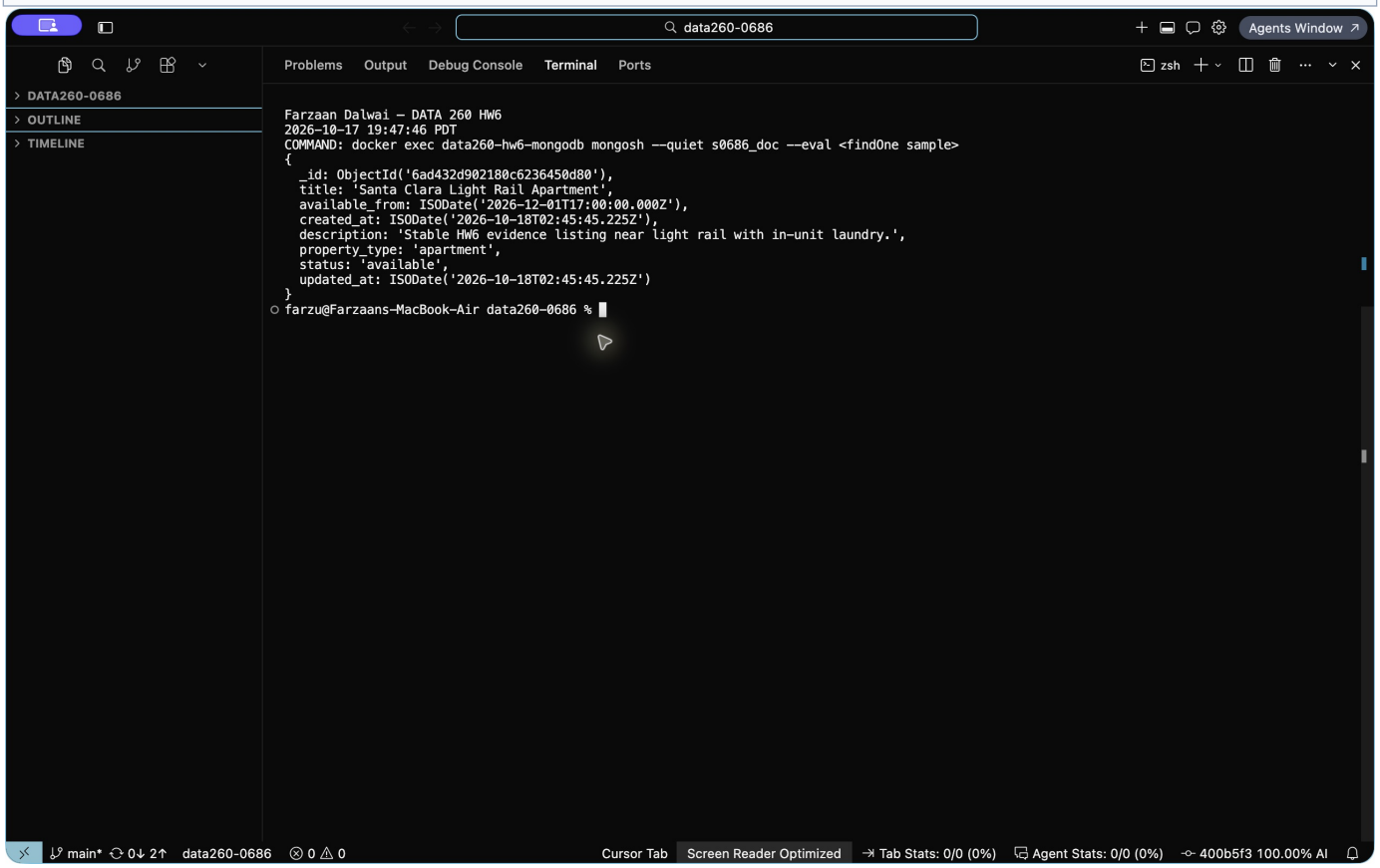


Figure 2. Real s0686_doc.rental_documents record shown with mongosh.

Error handling distinguishes malformed ObjectIds (400), missing records (404), request validation failures (422), and safe database failures (500). CORS and FastAPI JSON request parsing remain configured in the shared root application.

Part 2 — Short-, Long-, and Episodic Memory

The same MongoDB connection adds **messages**, **summaries**, and **episodes**, without changing the Part 1 collection. The default short window is N=12, summaries refresh every five user messages, and cosine retrieval returns the top three local MiniLM episode embeddings. The local Ollama adapter supplies assistant replies.

Live API surface

```
router = APIRouter(tags=['HW6 memory'])
@router.post('/api/chat', response_model=ChatResponse)
@router.get('/api/memory/{user_id}')
@router.get('/api/aggregate/{user_id}')
```

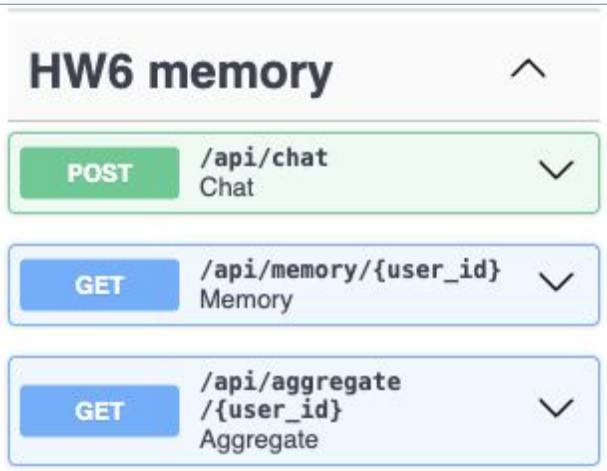


Figure 3. Browser view of the three live FastAPI HW6 memory endpoints.

Conversation evidence

```
user_id = 'demo-260686-memory'
for message in ten_fixed_messages:
    response = POST('/api/chat', user_id, session_id, message)
# Each turn saves user/reply and returns memory_used + usage.
```

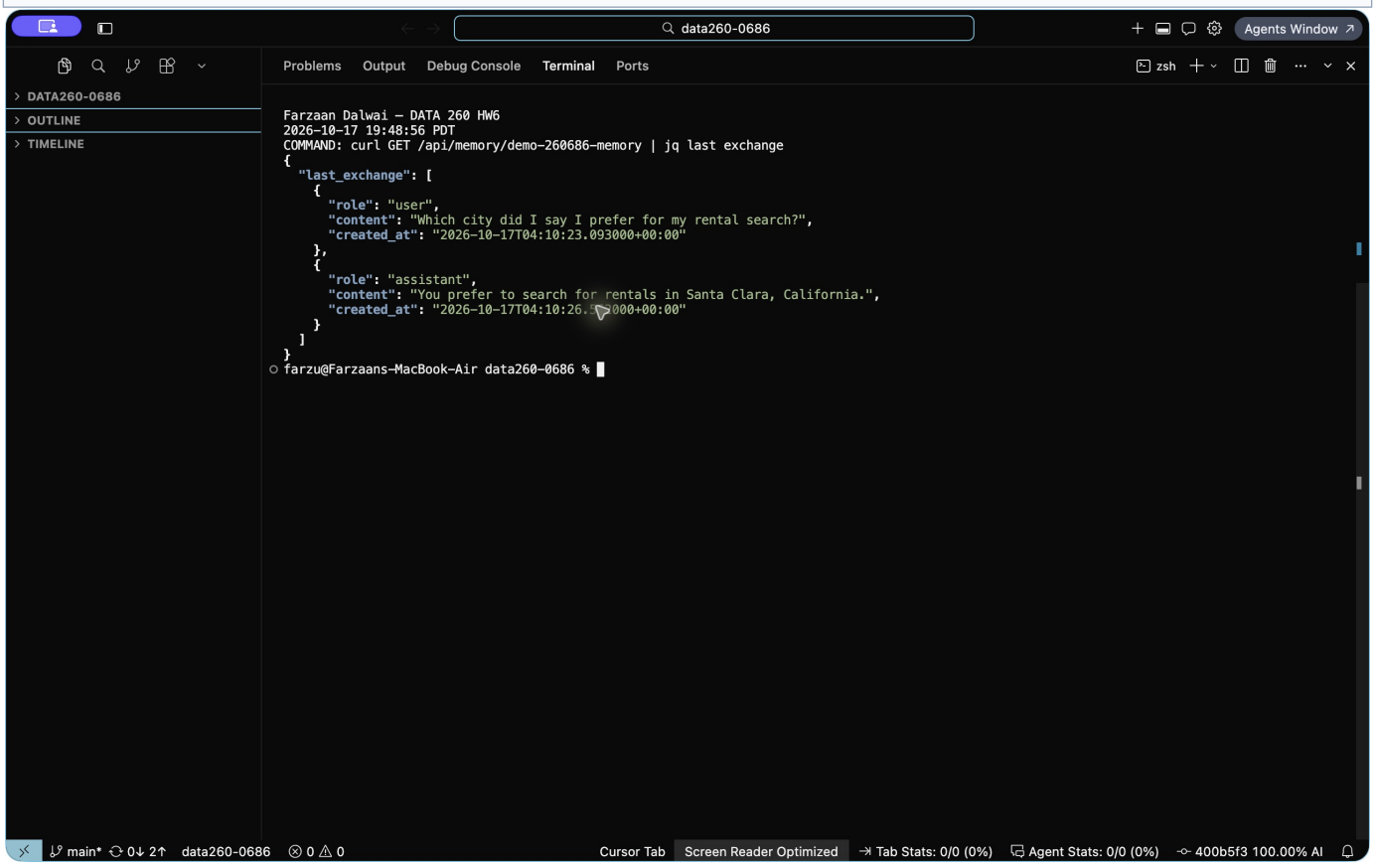


Figure 4. The persisted tenth exchange shows the local assistant recalling Santa Clara.

Memory endpoint evidence

```
def get_user_memory(user_id):
    messages = last_16_messages(user_id)
    return {'messages': messages,
      'latest_session_summary': latest_summary(user_id,'session'),
      'latest_lifetime_summary': latest_summary(user_id,'user'),
      'episodes': last_20_episodes(user_id)}
```

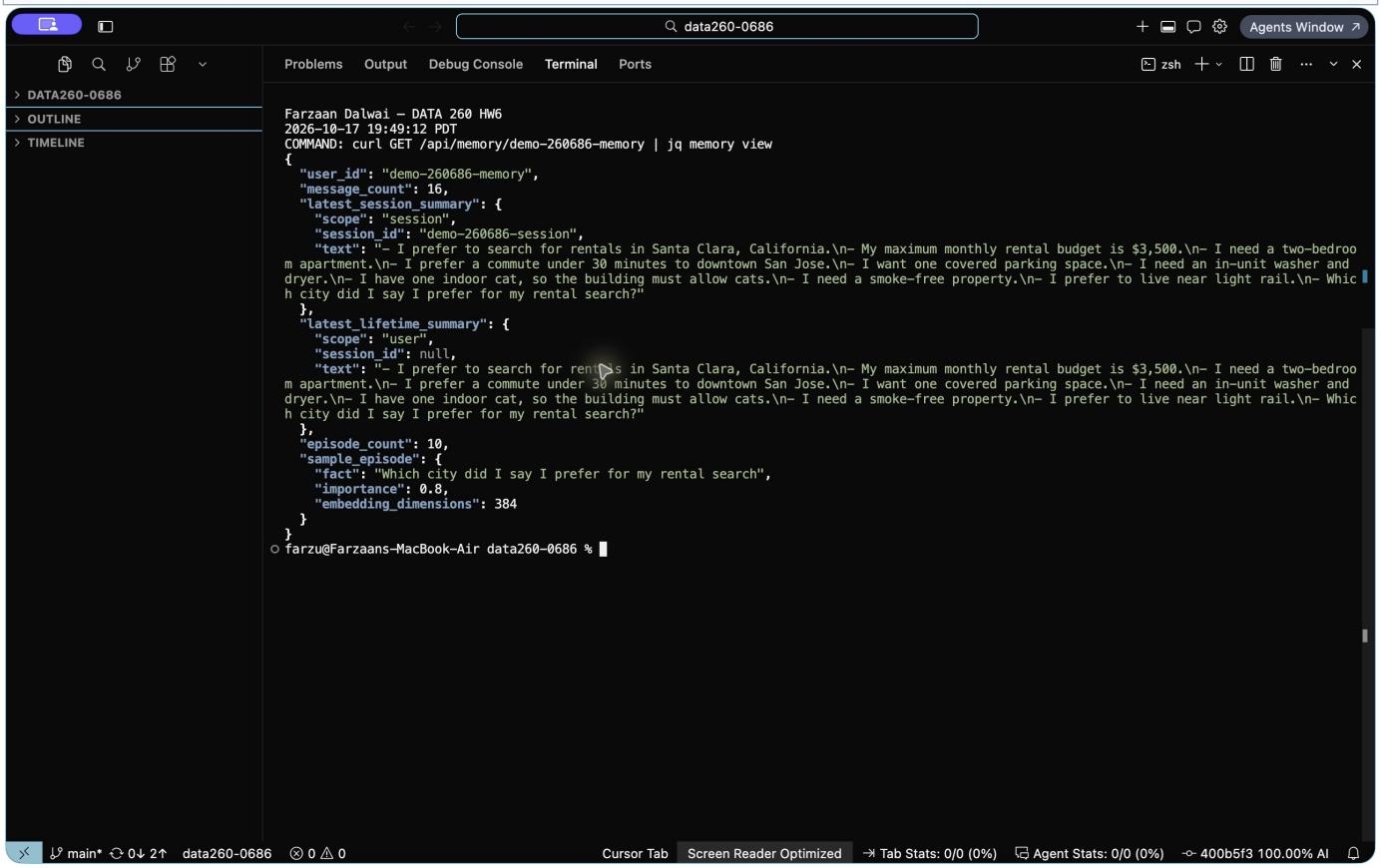


Figure 5. Live memory JSON: 16 messages, both summary scopes, 10 episodes, 384-dimensional embedding.

Aggregate endpoint evidence

```
pipeline = [
  {'$match': {'user_id': user_id}},
  {'$group': {'_id': {'$dateToString': {'format':'%Y-%m-%d',
      'date':'$created_at'}}, 'message_count': {'$sum':1}}}
]
```

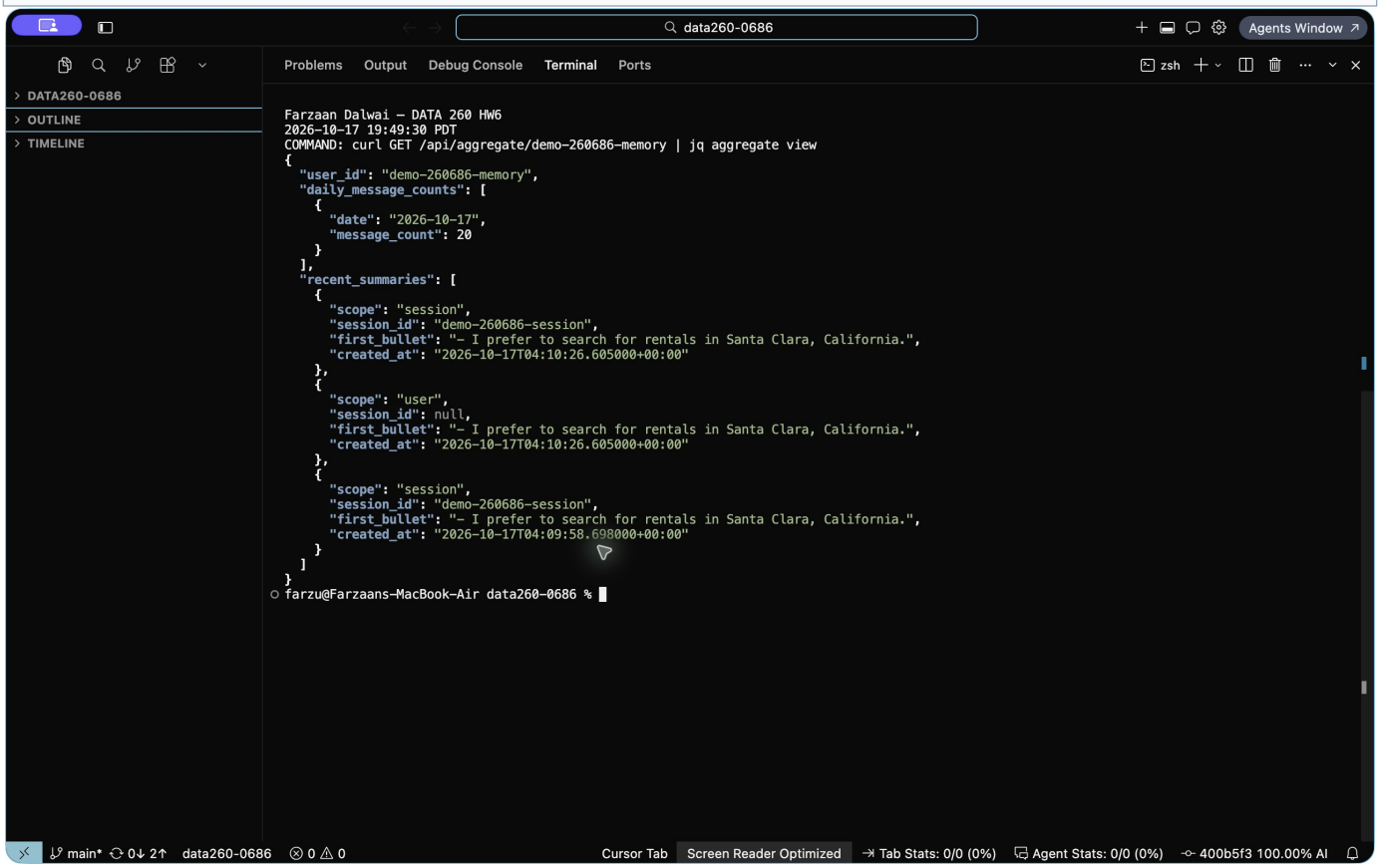


Figure 6. Live daily message counts and recent session/user summaries.

MongoDB memory collections

```
messages: user_id, session_id, role, content, created_at
summaries: user_id, nullable session_id, scope, text, created_at
episodes: user_id, session_id, fact, importance, embedding[], created_at
# Strict validators + compound query indexes are created idempotently.
```

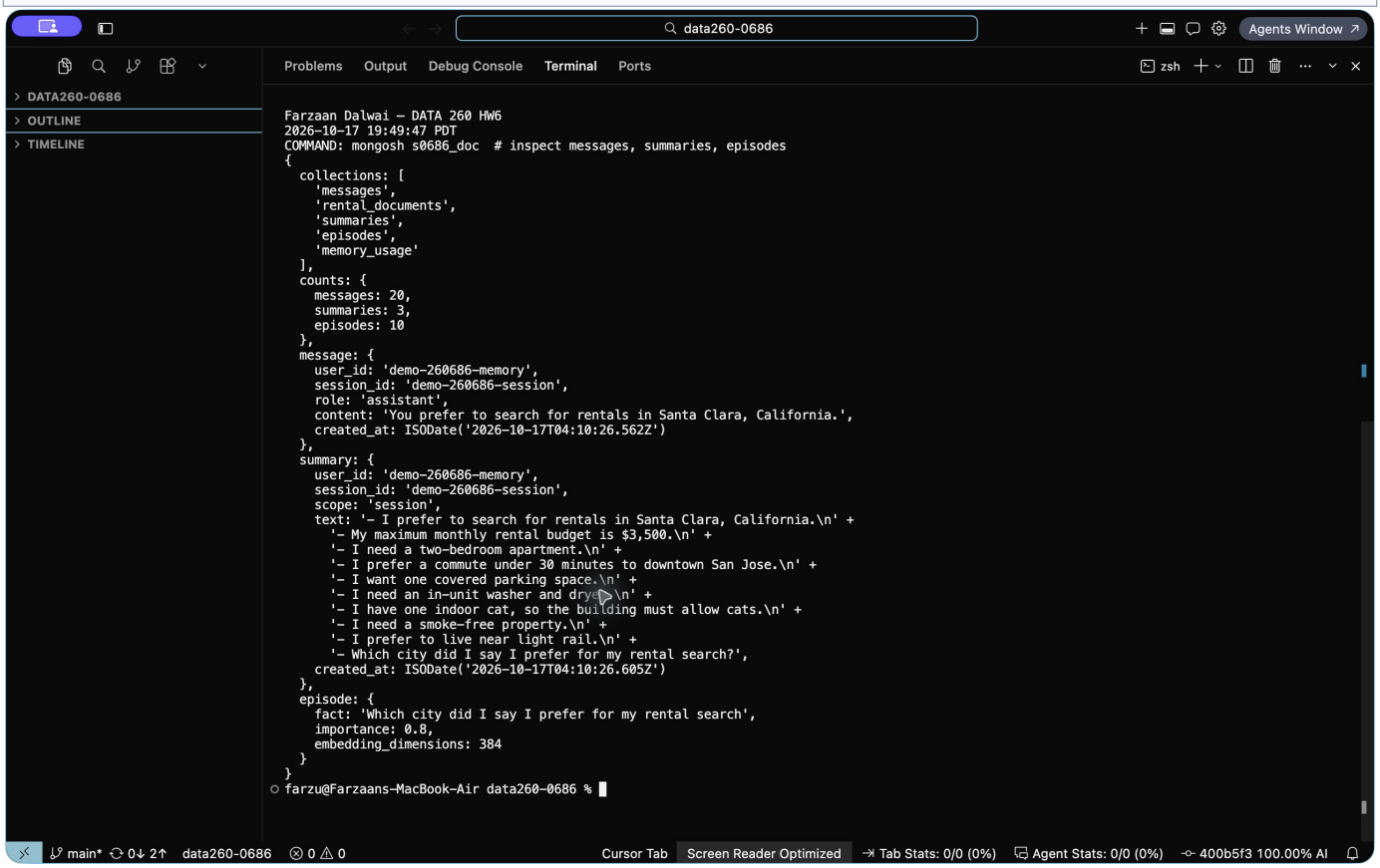


Figure 7. Live collections and demo documents: 20 messages, 3 summaries, 10 episodes.

Prompt assembly order is system primer, file memory when enabled, long-term summaries, short-term messages, retrieved episodes, and the current message. Episode extraction stores at most three explicit user facts with importance 0..1. Retrieval ranks up to 300 user episodes by cosine similarity and returns top-k. The ten-message milestone passed every objective check in raw/fast_milestone.json.

Part 3 — What Memory Costs

Each chat logs exact Ollama prompt/completion totals, MiniLM WordPiece component estimates for primer/summary/window/episodes, latency, identity, timestamp, condition, and runtime configuration. Five fixed probes and their complete 1–5 objective rubric are in memory_probes.yaml. Each condition used a fresh user/session, the identical 30-message conversation, and the same five probes; file memory was disabled to isolate MongoDB window behavior.

Measured results

```
CONDITIONS = [N=4, N=12, N=30, relevance_aware_eviction]
for condition in CONDITIONS:
    run_same_30_messages_in_fresh_session()
    score_same_five_probes_with_fixed_rubric()
# Exact totals: Ollama prompt_eval_count/eval_count; temperature=0.0
```

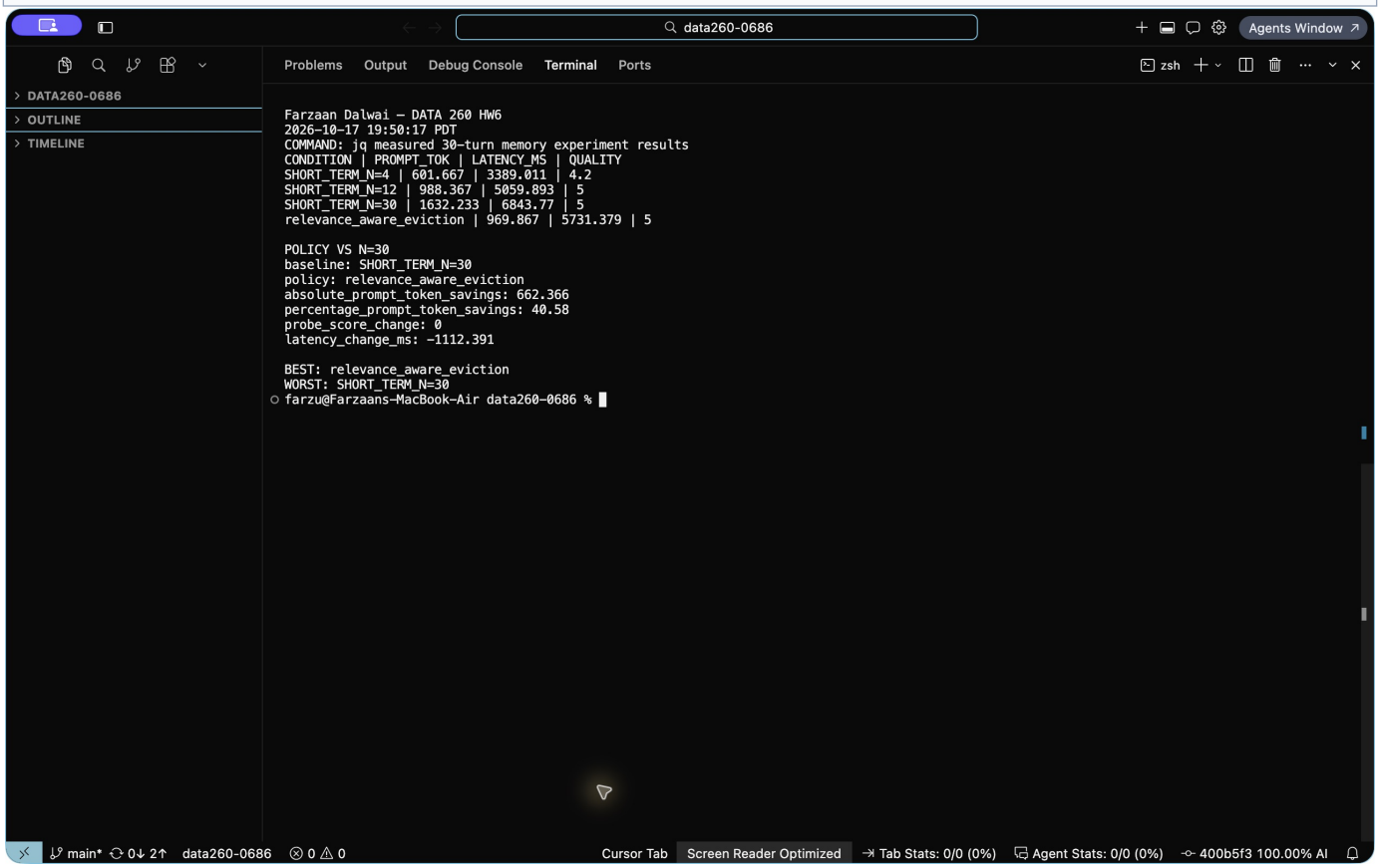


Figure 8. Corrected real local-model sweep and policy comparison.

Setting	Prompt tok/turn	Completion	Latency ms	Quality
SHORT_TERM_N=4	601.667	96.000	3389.011	4.200
SHORT_TERM_N=12	988.367	96.000	5059.893	5.000
SHORT_TERM_N=30	1632.233	96.000	6843.770	5.000
relevance_aware_eviction	969.867	96.000	5731.379	5.000

Relevance-aware eviction keeps the four newest messages plus the eight most semantically relevant older messages. Against N=30 it saved **662.366** prompt tokens/turn (**40.580%**) with quality change **+0.000** and latency change **-1112.391 ms**.

Setting	Input cost	Output cost	Total / 10,000 × 30
SHORT_TERM_N=4	$180.5001	$144.0000	$324.5001
SHORT_TERM_N=12	$296.5101	$144.0000	$440.5101
SHORT_TERM_N=30	$489.6699	$144.0000	$633.6699
relevance_aware_eviction	$290.9601	$144.0000	$434.9601

Selection rule: choose the lowest projected cost within 0.5 points of the best probe score; choose the highest cost in that same quality band as worst. Therefore **relevance_aware_eviction** is best and **SHORT_TERM_N=30** is worst. The first sweep was rejected because the default Ollama context silently truncated N=30 near 2,050 exact prompt tokens. After setting num_ctx=16384 and num_predict=96 consistently, N=30 grew past that ceiling; only corrected outputs are reported.

Part 4 — File-Based Memory and Consolidation

Root AGENT.md contains nine durable instructions spanning project description, stack, coding conventions, testing, and prohibited actions. Root MEMORY.md stores only durable facts and a last-consolidated timestamp. Loading accepts no caller path; commands can access only the two known root files.

Files, prompt, checks, and commands

```
def file_memory_prompt():
    return '=== DURABLE AGENT INSTRUCTIONS ===\n' + load_agent_memory() + \
           '\n=== DURABLE FACT MEMORY ===\n' + load_fact_memory()
def consolidation_required(now):
    return count_memory_lines() > 40 or now-last_consolidated > 24h
COMMANDS = {'/memory','/compact','/dream','/help'}
```

The final AGENT.md directs small typed functions, timezone-aware UTC, boundary validation, safe errors, objective tests, regressions, local-only models, preservation of prior data, and fixed memory paths. /memory displays; /compact consolidates; /dream appends up to three explicit durable preferences; /help documents all commands.

Consolidation evidence

```
if command == '/compact':
    result = compact_memory()
    return {'text': f"Compacted memory from {before} to {after} facts.",
            **result}
# Rank conflicts by confidence, then newest observation; retain alternatives.
```

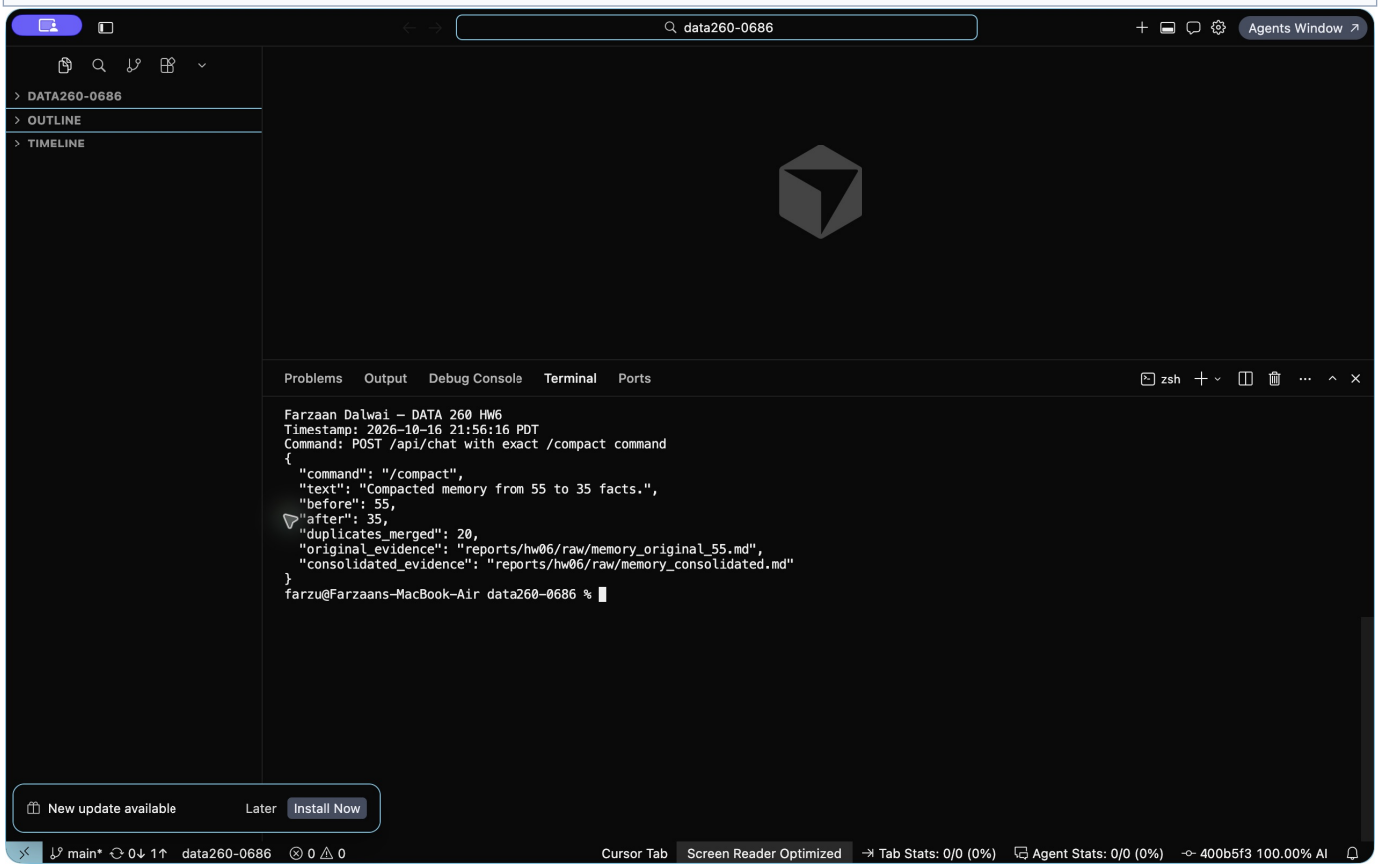


Figure 9. Real /compact output: 55 → 35, with 20 duplicate lines merged.

AGENT.md and contradiction verification

```
def verify_agent_compliance():
    checks = {'descriptive_snake_case_name': True, 'type_hints': True,
      'docstring': True, 'validates_external_input': True,
      'no_external_side_effects': True}
    return {'function':'normalize_rental_title','pass':all(checks.values())}
```

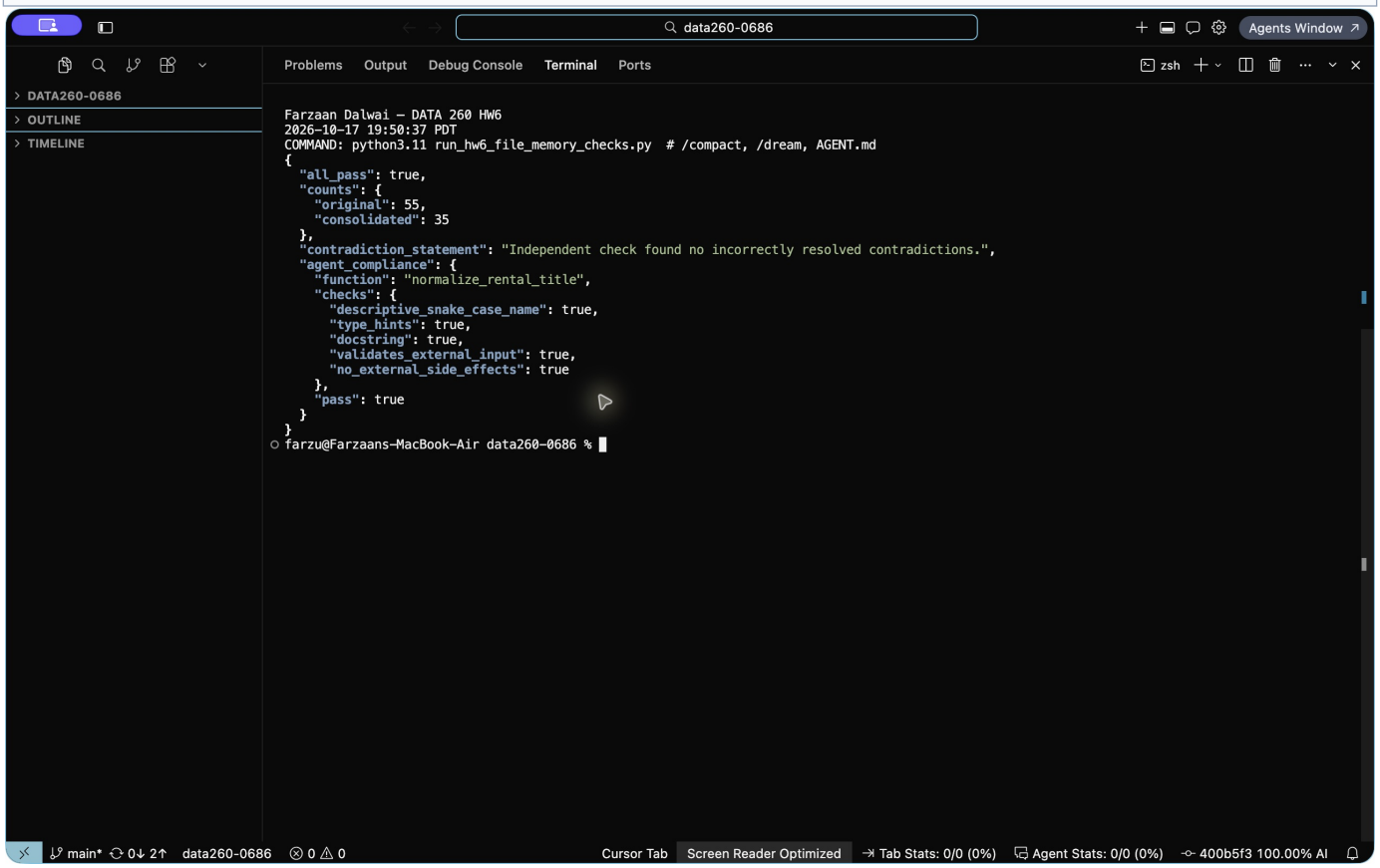


Figure 10. Real verification: all checks passed; no contradiction was resolved incorrectly.

The original exact 55-line state is in local commit **177192e**; the exact 35-line consolidated root state is in commit **9a7c2bd**. Raw copies preserve both versions. Deliberate conflicts were not silently deleted: each consolidated line retains alternatives and the explicit reason. Independent outcomes: preferred_city=Santa Clara, CA; max_monthly_budget=$3,500; pet_household=one indoor cat. Independent check found no incorrectly resolved contradictions.

Criterion	MongoDB memory	AGENT.md / MEMORY.md
Inspectability	Queryable structured documents; best for counts and filters.	Human-readable diff; easiest to audit in Git.
Restart persistence	Durable in the Mongo volume; independent of process.	Durable in repository files and commits.
Many sessions	Scales with indexed user/session/time queries.	Global file becomes noisy and needs consolidation.
Must-not-forget facts	Good for timestamped, scored, user-specific facts.	Best for a small set of critical instructions reviewed in Git.

Balanced choice: use MongoDB for high-volume, user/session-specific memories and retrieval; use file memory for a small auditable set of project rules and critical facts. Neither should overwrite the other without an explicit policy.

AI Use

1. What I used AI for and what I did

I used Codex to inspect the assignment and existing repository, implement and debug the FastAPI/PyMongo memory system, build objective test and experiment scripts, operate local applications, organize evidence, and draft this report. I supplied the repository, configuration, local services/model, assignment files, and authorization boundaries. All reported records, replies, metrics, hashes, and screenshots come from commands run on my Mac; no output was invented.

2–4. Unsuitable output, detection, and correction

The first 30-message-window experiment was unsuitable: exact Ollama prompt counts plateaued near 2,050 and the nominal N=30 condition had silently lost old context. I detected it by comparing per-turn prompt_eval_count across conditions. I preserved the invalid aggregate for audit, extended the shared adapter with explicit num_ctx/num_predict, and reran every condition in fresh sessions. The corrected N=30 prompts passed the former ceiling and the final files use only this sweep.

Verification and Repository Integrity

The self-check imports the shared app, calls the live backend on PORT_BASE, pings MongoDB, verifies the live validator, runs an HTTP CRUD round trip with cleanup, rejects invalid input, calls the local chat model, checks memory/aggregate endpoints, validates collections and embeddings, audits experiment rows/probes, checks both memory versions, and confirms original MySQL counts remain 5,000 listings / 200 notes / 1 manager.

```
python3.11 verify_hw06.py
# Objective smoke checks: FastAPI, MongoDB, CRUD, local chat, memory,
# aggregate, experiment rows/probes, file memory, and MySQL counts.
```

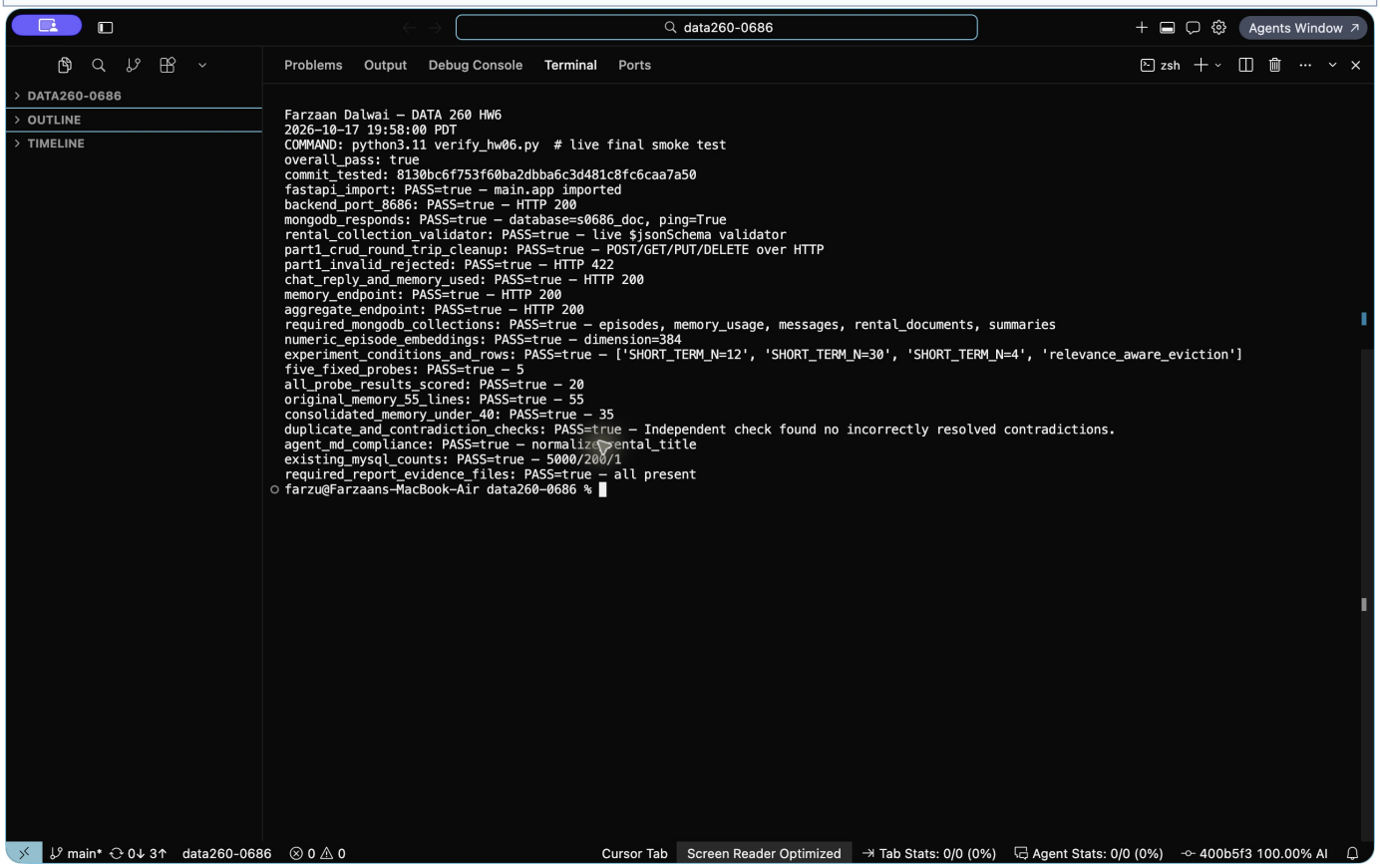


Figure 11. Final live self-check; verification.json contains the tested commit and each pass/fail result.

verification.json: overall_pass=**true**; 20/20 checks passed; tested commit **8130bc6f753f**.

Regression evidence: Vite production build passed, oxlint passed, both existing FastMCP servers imported, FastAPI imported with 17 routes, git diff --check passed, .env remains ignored, and no secret appears in screenshots or tracked evidence. The unrelated pre-existing reports/hw05/raw/agent_runs.jsonl modification was not staged or altered for HW6.

Submission status

Item	Status
Implementation and evidence	Complete locally and committed
Report and named submission copy	Generated and visually verified locally
GitHub push	Not performed — requires review/approval
Tag hw6	Not created — proposed after review
Required direct collaborators	Not confirmed; no access mutation performed

Evidence is reproducible from reports/hw06/RUN_LOG.txt, machine-readable raw files, verification.json, and the screenshot manifest. The only submission blockers are the deliberately withheld push/tag and missing direct collaborator membership.